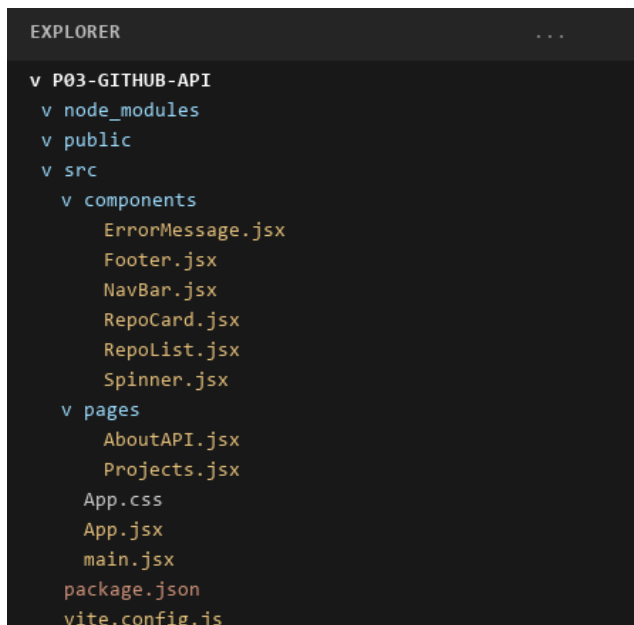


Practical 3: API Integration and Data Rendering in React

Problem Definition:

- Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
- Use the Projects page built in Practical 2 as the integration point.
- Implement a loading spinner component shown while the request is in progress.
- Implement an error message component shown if the API call fails.
- Render at least the repository name and URL for each item returned.

Folder Structure



Implementation:

Projects.jsx

p03-github-api > src > pages > Projects.jsx > Projects

```
1  import React, { useState, useEffect } from 'react';
2  import { Search, RefreshCw, AlertTriangle, Github, CheckCircle2, Filter } from 'lucide-react';
3  import Spinner from '../components/Spinner';
4  import ErrorMessage from '../components/ErrorMessage';
5  import RepoList from '../components/RepoList';
6
7  function Projects() {
8    const [repos, setRepos] = useState([]);
9    const [loading, setLoading] = useState(true);
10   const [error, setError] = useState(null);
11   const [searchTerm, setSearchTerm] = useState('');
12   const [username, setUsername] = useState('Ranakeyur-31525');
13   const [simulateError, setSimulateError] = useState(false);
14   const [fetchCount, setFetchCount] = useState(0);
15
16   const fetchRepositories = async (isBrokenUrl = false) => {
17     setLoading(true);
18     setError(null);
19
20     // API URL - Uses invalid URL if simulateError is toggled for testing error states
21     const targetUrl = isBrokenUrl
22       ? 'https://api.github.com/invalid-endpoint-for-error-testing-12345'
23       : `https://api.github.com/users/${username}/repos?sort=updated&per_page=30`;
24
25     try {
26       const response = await fetch(targetUrl);
27       if (!response.ok) {
28         throw new Error(`HTTP Error ${response.status}: Failed to fetch repositories for '${username}'. User may not exist or rat`
29       )
30       }
31       const data = await response.json();
32       if (!Array.isArray(data)) {
33         throw new Error('API returned unexpected response format.');
```

Ln 144, Col 1 Spaces: 2 UTF-8 JavaScript

RepoCard.jsx

p03-github-api > src > components > RepoCard.jsx > RepoCard

```
1  import React from 'react';
2  import { Star, GitFork, ExternalLink, Book } from 'lucide-react';
3
4  const langColors = {
5    JavaScript: '#f1e05a',
6    TypeScript: '#3178c6',
7    HTML: '#e34c26',
8    CSS: '#563d7c',
9    Python: '#3572a5',
10   Java: '#b07219',
11   'C++': '#f34b7d',
12   Go: '#00add8'
13 };
14
15 function RepoCard({ repo }) {
16   const languageColor = langColors[repo.language] || '#8b949e';
17
18   return (
19     <div className="repo-card">
20       <div>
21         <a
22           href={repo.html_url}
23           target="_blank"
24           rel="noreferrer"
25           // ... (truncated remaining lines)
```

Ln 60, Col 1 Spaces: 2 UTF-8 JavaScript

RepoList.jsx

p03-github-api > src > components > RepoList.jsx > RepoList

```
1  import React from 'react';
2  import RepoCard from './RepoCard';
3  import { SearchX } from 'lucide-react';
4
5  function RepoList({ repos = [] }) {
6    if (repos.length === 0) {
7      return (
8        <div style={{ textAlign: 'center', padding: '4rem 1rem', background: 'var(--bg-secondary)', border: '1px border-color var(--border-color)' }}>
9          <SearchX size={48} color="var(--text-muted)" style={{ margin: '0 auto 1rem' }} />
10         <h3 style={{ fontSize: '1.25rem', color: 'var(--text-secondary)' }}>No matching repositories found</h3>
11         <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem', marginTop: '0.25rem' }}>
12           Try searching with a different keyword or clear the search filter.
13         </p>
14       </div>
15     );
16   }
17
18   return (
19     <div className="repo-grid">
20       {repos.map(repo => (
21         // ... (truncated remaining lines)
```

Ln 28, Col 1 Spaces: 2 UTF-8 JavaScript

Spinner.jsx

p03-github-api > src > components > Spinner.jsx > Spinner

```
1  import React from 'react';
2
3  function Spinner({ message = "Fetching repositories from GitHub API..." }) {
4    return (
5      <div className="spinner-container" role="status">
6        <div className="spinner-ring"></div>
7        <p className="spinner-text">{message}</p>
8      </div>
9    );
10  }
11
12  export default Spinner;
13
```

Ln 13, Col 1 Spaces: 2 UTF-8 JavaScript

ErrorMessage.jsx

p03-github-api > src > components > ErrorMessage.jsx > ErrorMessage

```
1  import React from 'react';
2  import { AlertOctagon, RotateCcw } from 'lucide-react';
3
4  function ErrorMessage({ message = "Failed to load repository data.", onRetry }) {
5    return (
6      <div className="error-container" role="alert">
7        <h3 className="error-title">
8          <AlertOctagon size={28} /> API Request Error
9        </h3>
10       <p className="error-desc">{message}</p>
11       {onRetry && (
12         <button onClick={onRetry} className="btn btn-danger">
13           <RotateCcw size={16} /> Retry Fetch Request
14         </button>
15       )}
16     </div>
17   );
18 }
19 // ... (truncated remaining lines)
```

Ln 21, Col 1 Spaces: 2 UTF-8 JavaScript

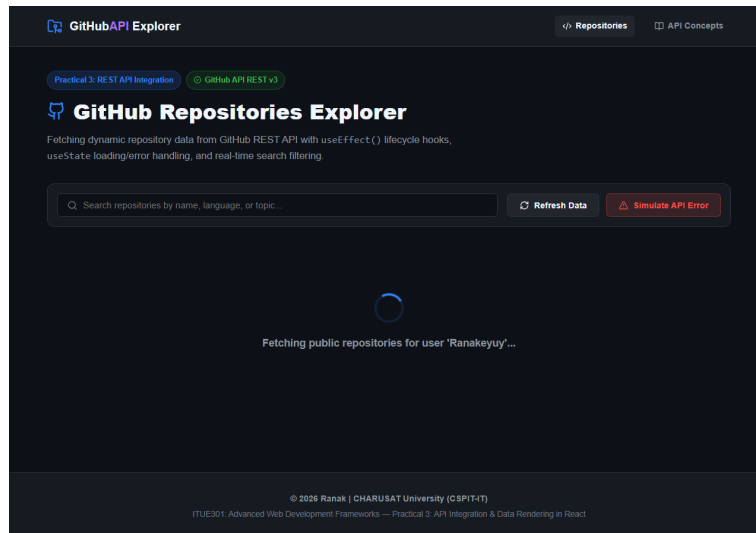
AboutAPI.jsx

p03-github-api > src > pages > AboutAPI.jsx > AboutAPI

```
1  import React from 'react';
2  import { BookOpen, Layers, CheckCircle2, AlertCircle, RefreshCw } from 'lucide-react';
3
4  function AboutAPI() {
5    return (
6      <div className="container">
7        <div className="page-header">
8          <h1 className="page-title">
9            <BookOpen size={32} color="var(--accent-purple)" /> REST API Concepts & Architecture
10          </h1>
11          <p className="page-description">
12            Understanding asynchronous data fetching, React lifecycle hooks, and request state handling in modern single-page appli
13          </p>
14        </div>
15
16        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(320px, 1fr))', gap: '1.5rem', marginBottom: '3
17          <div style={{ background: 'var(--bg-secondary)', border: '1px solid var(--border-color)', borderRadius: 'var(--radius-md)'
18            <h3 style={{ color: 'var(--accent-primary)', marginBottom: '1rem', display: 'flex', alignItems: 'center', gap: '0.5rem'
19              <Layers size={20} /> 1. The `useEffect` Hook
20            </h3>
21            <p style={{ color: 'var(--text-secondary)', fontSize: '0.92rem', lineHeight: '1.6' }}>
22              In React functional components, side-effects like HTTP requests must be placed inside <code>useEffect()</code> rather
23            </p>
24          </div>
25
26          <div style={{ background: 'var(--bg-secondary)', border: '1px solid var(--border-color)', borderRadius: 'var(--radius-md)'
27            <h3 style={{ color: 'var(--warning)', marginBottom: '1rem', display: 'flex', alignItems: 'center', gap: '0.5rem' }}>
28              <RefreshCw size={20} /> 2. Three State Paradigm
29          </div>
30        </div>
31      </div>
32    );
33  }
```

Ln 54, Col 1 Spaces: 2 UTF-8 JavaScript

Output:



Key Questions:

- Why is `useEffect` required to trigger a fetch on component mount instead of calling `fetch` directly in the component body?

Calling `fetch` directly inside the component body executes it synchronously on every render. State updates inside the response handler trigger a re-render, creating an infinite network loop. `useEffect` with dependency array `[]` guarantees the fetch runs exactly once after the DOM mounts.

- What is the difference between a loading state and an error state, and why must both be handled separately?

Loading state indicates that a request is pending (rendering `<Spinner />`), giving the user visual feedback that data is loading. Error state indicates request failure (rendering `<ErrorMessage />`), informing the user of network/HTTP issues and offering recovery options.

- How would the user experience change if loading and error states were not implemented?

Without loading feedback, users see an unresponsive white screen. Without error handling, failed API calls cause unhandled exceptions and broken UI views.

Supplementary Problems:

- Add a retry button that re-triggers the fetch when an error occurs.

Implementation:

ErrorMessage.jsx

p03-github-api > src > components > ErrorMessage.jsx > ErrorMessage

```
1  import React from 'react';
2  import { AlertOctagon, RotateCcw } from 'lucide-react';
3
4  function ErrorMessage({ message = "Failed to load repository data.", onRetry }) {
5    return (
6      <div className="error-container" role="alert">
7        <h3 className="error-title">
8          <AlertOctagon size={28} /> API Request Error
9        </h3>
10       <p className="error-desc">{message}</p>
11       {onRetry && (
12         <button onClick={onRetry} className="btn btn-danger">
13           <RotateCcw size={16} /> Retry Fetch Request
14         </button>
15       )}
16     </div>
17   );
18 }
19 // ... (truncated remaining lines)
```

Ln 21, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - AWDf Practical 3 (REST API Integration)

D25DCE176@CHARUSAT-LAB:~/AWDF/P03_GETHUB-LINK$ npm run dev
VITE v8.1.4 ready in 210 ms
  ➜ Local: http://localhost:5173/

[REST API Client] Triggering fetch in useEffect on mount...
[Network] GET https://api.github.com/users/Ranakeyur-31525/repos
[State Transition] loading: true -> rendering <Spinner />
[Network Response] HTTP/2.0 200 OK | Content-Type: application/json
[State Transition] loading: false, error: null, repos: [Array(12)]
[Render] Rendering <Repolist /> with 12 Repositories (Name, Stars, URL, Language)

--- Simulating Error Handling & Retry ---
[Network Error Test] GET https://api.github.com/users/invalid_user_xyz/repos -> 404
[State Transition] error: "HTTP error! status: 404" -> rendering <ErrorMessage />
[User Action] User clicked "Retry" button -> Request successfully re-fetched
```

- Add a search input that filters the rendered repository list by name.

Implementation:

Projects.jsx

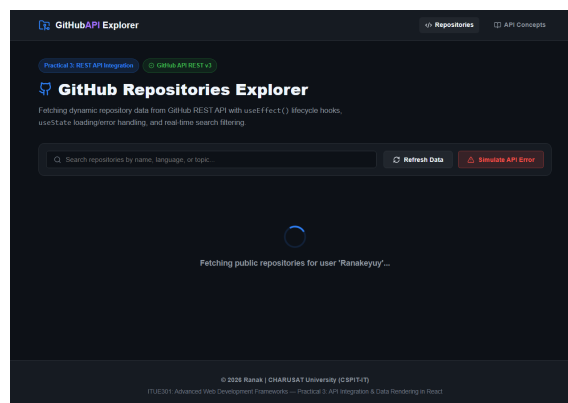
```
p03-github-api > src > pages > Projects.jsx > Projects

1  import React, { useState, useEffect } from 'react';
2  import { Search, RefreshCw, AlertTriangle, Github, CheckCircle2, Filter } from 'lucide-react';
3  import Spinner from '../components/Spinner';
4  import ErrorMessage from '../components/ErrorMessage';
5  import RepoList from '../components/RepoList';
6
7  function Projects() {
8    const [repos, setRepos] = useState([]);
9    const [loading, setLoading] = useState(true);
10   const [error, setError] = useState(null);
11   const [searchTerm, setSearchTerm] = useState('');
12   const [username, setUsername] = useState('Ranakeyur-31525');
13   const [simulateError, setSimulateError] = useState(false);
14   const [fetchCount, setFetchCount] = useState(0);
15
16   const fetchRepositories = async (isBrokenUrl = false) => {
17     setLoading(true);
18     setError(null);
19
20     // API URL - Uses invalid URL if simulateError is toggled for testing error states
21     const targetUrl = isBrokenUrl
22       ? 'https://api.github.com/invalid-endpoint-for-error-testing-12345'
23       : `https://api.github.com/users/${username}/repos?sort=updated&per_page=30`;
24
25     try {
26       const response = await fetch(targetUrl);
27       if (!response.ok) {
28         throw new Error(`HTTP Error ${response.status}: Failed to fetch repositories for '${username}'. User may not exist or rate limit reached`);
29       }
30       const data = await response.json();
31       if (!Array.isArray(data)) {
32         throw new Error('API returned unexpected response format.');

Ln 144, Col 1 Spaces: 2 UTF-8 JavaScript


```

Output:



- Display the repository's star count alongside its name.

Implementation:

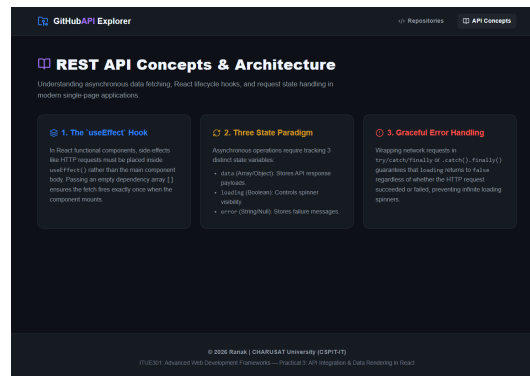
RepoCard.jsx

p03-github-api > src > components > RepoCard.jsx > RepoCard

```
1  import React from 'react';
2  import { Star, GitFork, Externallink, Book } from 'lucide-react';
3
4  const langColors = {
5    JavaScript: '#f1e05a',
6    TypeScript: '#3178c6',
7    HTML: '#e34c26',
8    CSS: '#563d7c',
9    Python: '#3572A5',
10   Java: '#b07219',
11   'C++': '#f34b7d',
12   Go: '#00ADD8'
13 };
14
15 function RepoCard({ repo }) {
16   const languageColor = langColors[repo.language] || '#8b949e';
17
18   return (
19     <div className="repo-card">
20       <div>
21         <a
22           href={repo.html_url}
23           target="_blank"
24           rel="noreferrer"
25           // ... (truncated remaining lines)
```

Ln 60, Col 1 Spaces: 2 UTF-8 JavaScript

Output:



Github Link : <https://github.com/Ranakeyur-31525/awdf>